

# GROUP BY CLAUSE ASSIGNMENT

## SQL Questions Q51–Q100

```
-- =====
-- SECTION 2 – GROUP BY CLAUSE
-- =====

-- Q51. Count the number of customers in each city.
SELECT city, COUNT(*) AS total_customers
FROM customers
GROUP BY city;

-- Q52. Find the number of customers who signed up on each date.
SELECT signup_date, COUNT(*) AS total_customers
FROM customers
GROUP BY signup_date;

-- Q53. Find the number of customers in each city who signed up after '2023-01-01'.
SELECT city, COUNT(*) AS total_customers
FROM customers
WHERE signup_date > '2023-01-01'
GROUP BY city;

-- Q54. Find the latest signup date for each city.
SELECT city, MAX(signup_date) AS latest_signup_date
FROM customers
GROUP BY city;

-- Q55. Find the earliest signup date for each city.
SELECT city, MIN(signup_date) AS earliest_signup_date
FROM customers
GROUP BY city;

-- Q56. Count customers in each city and display only cities having more than 5 customers.
SELECT city, COUNT(*) AS total_customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 5;

-- Q57. Count customers in each city and display only cities having more than 10 customers.
SELECT city, COUNT(*) AS total_customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 10;

-- Q58. Count customers in each city after the year 2023 and display cities having more than 5 customers.
SELECT city, COUNT(*) AS total_customers
FROM customers
WHERE signup_date >= '2023-01-01'
GROUP BY city
HAVING COUNT(*) > 5;

-- Q59. Count customers in each city after the year 2024.
SELECT city, COUNT(*) AS total_customers
FROM customers
WHERE signup_date >= '2024-01-01'
GROUP BY city;

-- Q60. Find the number of customers for each city and signup year.
SELECT city, YEAR(signup_date) AS signup_year, COUNT(*) AS total_customers
FROM customers
GROUP BY city, YEAR(signup_date);

-- Q61. Count the number of products in each category.
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id;

-- Q62. Count the number of products for each brand.
SELECT brand_id, COUNT(*) AS total_products
FROM products
```

```
GROUP BY brand_id;
```

```
-- Q63. Find the average price of products in each category.
SELECT category_id, AVG(price) AS average_price
FROM products
GROUP BY category_id;
```

```
-- Q64. Find the average price of products for each brand.
SELECT brand_id, AVG(price) AS average_price
FROM products
GROUP BY brand_id;
```

```
-- Q65. Find the minimum price of products in each category.
SELECT category_id, MIN(price) AS minimum_price
FROM products
GROUP BY category_id;
```

```
-- Q66. Find the maximum price of products in each category.
SELECT category_id, MAX(price) AS maximum_price
FROM products
GROUP BY category_id;
```

```
-- Q67. Find the minimum price of products for each brand.
SELECT brand_id, MIN(price) AS minimum_price
FROM products
GROUP BY brand_id;
```

```
-- Q68. Find the maximum price of products for each brand.
SELECT brand_id, MAX(price) AS maximum_price
FROM products
GROUP BY brand_id;
```

```
-- Q69. Find the total stock available in each category.
SELECT category_id, SUM(stock) AS total_stock
FROM products
GROUP BY category_id;
```

```
-- Q70. Find the total stock available for each brand.
SELECT brand_id, SUM(stock) AS total_stock
FROM products
GROUP BY brand_id;
```

```
-- Q71. Find the average stock available in each category.
SELECT category_id, AVG(stock) AS average_stock
FROM products
GROUP BY category_id;
```

```
-- Q72. Find the average stock available for each brand.
SELECT brand_id, AVG(stock) AS average_stock
FROM products
GROUP BY brand_id;
```

```
-- Q73. Find categories having more than 2 products.
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) > 2;
```

```
-- Q74. Find brands having more than 2 products.
SELECT brand_id, COUNT(*) AS total_products
FROM products
GROUP BY brand_id
HAVING COUNT(*) > 2;
```

```
-- Q75. Find categories whose average product price is greater than 10000.
SELECT category_id, AVG(price) AS average_price
FROM products
GROUP BY category_id
HAVING AVG(price) > 10000;
```

```
-- Q76. Find brands whose average product price is greater than 10000.
```

```

SELECT brand_id, AVG(price) AS average_price
FROM products
GROUP BY brand_id
HAVING AVG(price) > 10000;

-- Q77. Find categories having total stock greater than 500.
SELECT category_id, SUM(stock) AS total_stock
FROM products
GROUP BY category_id
HAVING SUM(stock) > 500;

-- Q78. Find brands having total stock greater than 500.
SELECT brand_id, SUM(stock) AS total_stock
FROM products
GROUP BY brand_id
HAVING SUM(stock) > 500;

-- Q79. Count the number of orders placed by each customer.
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id;

-- Q80. Find the total order amount for each customer.
SELECT customer_id, SUM(total_amount) AS total_amount
FROM orders
GROUP BY customer_id;

-- Q81. Find the average order amount for each customer.
SELECT customer_id, AVG(total_amount) AS average_amount
FROM orders
GROUP BY customer_id;

-- Q82. Find the minimum order amount for each customer.
SELECT customer_id, MIN(total_amount) AS minimum_amount
FROM orders
GROUP BY customer_id;

-- Q83. Find the maximum order amount for each customer.
SELECT customer_id, MAX(total_amount) AS maximum_amount
FROM orders
GROUP BY customer_id;

-- Q84. Count the number of orders for each order status.
SELECT order_status, COUNT(*) AS total_orders
FROM orders
GROUP BY order_status;

-- Q85. Find the total order amount for each order status.
SELECT order_status, SUM(total_amount) AS total_amount
FROM orders
GROUP BY order_status;

-- Q86. Find the average order amount for each order status.
SELECT order_status, AVG(total_amount) AS average_amount
FROM orders
GROUP BY order_status;

-- Q87. Find order statuses having more than 5 orders.
SELECT order_status, COUNT(*) AS total_orders
FROM orders
GROUP BY order_status
HAVING COUNT(*) > 5;

-- Q88. Find order statuses having a total order amount greater than 50000.
SELECT order_status, SUM(total_amount) AS total_amount
FROM orders
GROUP BY order_status
HAVING SUM(total_amount) > 50000;

-- Q89. Find the total order amount for each customer for orders above 1000.
SELECT customer_id, SUM(total_amount) AS total_amount
FROM orders

```

```
WHERE total_amount > 1000
GROUP BY customer_id;
```

```
-- Q90. Find the number of orders for each customer placed after '2025-01-01'.
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
WHERE order_date > '2025-01-01'
GROUP BY customer_id;
```

```
-- Q91. Count the number of order items for each order.
SELECT order_id, COUNT(*) AS total_items
FROM order_items
GROUP BY order_id;
```

```
-- Q92. Find the total quantity ordered for each order.
SELECT order_id, SUM(quantity) AS total_quantity
FROM order_items
GROUP BY order_id;
```

```
-- Q93. Find the average quantity ordered for each order.
SELECT order_id, AVG(quantity) AS average_quantity
FROM order_items
GROUP BY order_id;
```

```
-- Q94. Find the total discount given for each order.
SELECT order_id, SUM(discount) AS total_discount
FROM order_items
GROUP BY order_id;
```

```
-- Q95. Find the average discount for each order.
SELECT order_id, AVG(discount) AS average_discount
FROM order_items
GROUP BY order_id;
```

```
-- Q96. Find the total quantity sold for each product.
SELECT product_id, SUM(quantity) AS total_quantity
FROM order_items
GROUP BY product_id;
```

```
-- Q97. Find products having total quantity sold greater than 2.
SELECT product_id, SUM(quantity) AS total_quantity
FROM order_items
GROUP BY product_id
HAVING SUM(quantity) > 2;
```

```
-- Q98. Count the number of payments for each payment method.
SELECT payment_method, COUNT(*) AS total_payments
FROM payments
GROUP BY payment_method;
```

```
-- Q99. Count the number of payments for each payment status.
SELECT payment_status, COUNT(*) AS total_payments
FROM payments
GROUP BY payment_status;
```

```
-- Q100. Find payment methods having more than 5 payments.
SELECT payment_method, COUNT(*) AS total_payments
FROM payments
GROUP BY payment_method
HAVING COUNT(*) > 5;
```